```
  return http
    .authorizeExchange(exchange ->
      exchange.anyExchange().authenticated())     ❶
    .build();
}
```

❶ All requests require authentication.

Spring Security provides several authentication strategies, including HTTP Basic, login form, SAML, and OpenID Connect. For this example, we want to use the login form strategy, which we can enable via the `formLogin()` method exposed by the `ServerHttpSecurity` object. We'll go with the default configuration (available via the Spring Security `Customizer` interface), which includes a login page, provided by the framework out of the box, and an automatic redirect to that page whenever a request is not authenticated:

```
@Bean
SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
  return http
    .authorizeExchange(exchange -> exchange.anyExchange().authenticated())
    .formLogin(Customizer.withDefaults())      ❶
    .build();
}
```

❶ Enables user authentication via a login form

Next, annotate the `SecurityConfig` class with `@EnableWebFluxSecurity` to enable Spring Security WebFlux support. The final security configuration is shown in the following listing.

Listing 11.2 Requiring authentication for all endpoints via a login form

```
package com.polarbookshop.edgeservice.config;

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.reactive.
➥EnableWebFluxSecurity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
```